Test Data Selection Based on Applying Mutation Testing to Decision Tree Models

Beatriz N. C. Siveira
ICMC/USP
Universidade de São Paulo
São Carlos, SP, Brazil
beatrizncs@usp.br

Vinicius H. S. Durelli
UFSJ
Universidade Federal São João del Rei
São João del Rei, MG, Brazil
durelli@ufsj.edu.br

Sebastião H. N. Santos
ICMC/USP
Universidade de São Paulo
São Carlos, SP, Brazil
sebastiaohns@usp.br

Rafael S. Durelli
UFLA
Universidade Federal de Lavras
Lavras, MG, Brazil
rafael.durelli@ufla.br

Marcio E. Delamaro
ICMC/USP
Universidade de São Paulo
São Carlos, SP, Brazil
delamaro@icmc.usp.br

Simone R. S. Souza
ICMC/USP
Universidade de São Paulo
São Carlos, SP, Brazil
srocio@icmc.usp.br

ABSTRACT

Software testing is crucial to ensure software quality, verifying that it behaves as expected. This activity plays a crucial role in identifying defects from the early stages of the development process. Software testing is especially essential in complex or critical systems, such as those using Machine Learning (ML) techniques, since the models can present uncertainties and errors that affect their reliability. This work investigates the use of mutation testing to support the validation of ML applications. Our approach involves applying mutation analysis to the decision tree structure. The resulting mutated trees are a reference for selecting a test dataset that can effectively identify incorrect classifications in machine learning models. Preliminary results suggest that the proposed approach can successfully improve the test data selection for ML applications.

CCS CONCEPTS

• **Software and its engineering → Software testing and debugging**.

KEYWORDS

Software Testing, Machine Learning, Decision Tree, Mutation Testing

ACM Reference Format:
Beatriz N. C. Siveira, Vinicius H. S. Durelli, Sebastião H. N. Santos, Rafael S. Durelli, Marcio E. Delamaro, and Simone R. S. Souza. 2023. Test Data Selection Based on Applying Mutation Testing to Decision Tree Models. In *8th Brazilian Symposium on Systematic and Automated Software Testing (SAST 2023), September 25–29, 2023, Campo Grande, MS, Brazil.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3624032.3624038

SAST 2023, September 25–29, 2023, Campo Grande, MS, Brazil

ACM ISBN 979-8-4007-1629-4/23/09…$15.00
https://doi.org/10.1145/3624032.3624038

1 INTRODUCTION

In recent years, due to the growing abundance of available data, machine learning (ML) algorithms have emerged as an alternative data-driven approach for solving a myriad of problems [3, 6]. Given the escalating popularity of ML algorithms, researchers and practitioners alike have been exploring ways to evaluate and improve the reliability and quality of ML-based software systems [4, 23].

Before deploying ML models, testers need to approach these models like conventional software and carry out testing efforts to uncover potential issues. This testing phase is an essential part of the training and deployment process. However, despite its importance, uncovering problems in ML-based applications presents significant challenges. Many challenges arise due to the inherent differences between traditional software and ML-based software, which is more statistically-oriented and inherently less deterministic. The behavior of ML-based software is derived from a data-driven process: ML algorithms are used to model and understand complex datasets, and the process of deriving decision logic from data through training can vary greatly depending on the specific ML algorithm employed. To address the challenges associated with testing ML-based systems, researchers have started adopting proven methods and approaches, such as mutation testing. Drawing inspiration from white-box testing approaches, researchers have been probing into how the internal structure of ML models can be used to generate test cases. However, it is worth noting that the lion's share of current testing techniques are not directly applicable to software representations of ML models.

In conventional software testing settings, mutation testing involves repeatedly making subtle changes in the form of small syntactic deviations (i.e., *mutations*) to the software being tested. The purpose of this is to examine if any test case fails when executed against the mutated program. A mutated program is referred to as a *mutant* and is said to be *killed* when at least one test case detects the introduced mutation (i.e., fails due to the mutation). When a test case is sensitive enough to kill a mutant, the main implication is that the test suite is effective. Thus, the primary goal of mutation testing is to evaluate the quality of the test suite by fostering the creation of test cases that uncover problems that stem from the mutants (i.e., the small syntactic deviations made to the program under test). Mutation testing has garnered significant research attention

since its inception. It has been employed for many purposes and, in recent years, significant strides have been made to enhance its computational efficiency, making it a valuable approach to testing software systems. We propose an approach that capitalizes on mutation testing to create effective test suites for uncovering faults in ML models. Specifically, our approach is centered around mutating some elements of the internal structure of decision tree models, allowing for the selection of inputs (i.e., test cases) that are sensitive enough to reveal discrepancies in the ensuing predictions (i.e., outcomes).

Decision tree is a family of ML algorithms that yield graphical models that bear a resemblance to trees, making them easily comprehensible to humans [12]. These algorithms partition the predictor space into unique, non-overlapping parts. The criteria used for this partitioning are embodied in a model that mirrors a tree structure. The graphical representation of the resulting tree model resembles a flowchart like structure, where leaf nodes indicate the result of a series of sections, while interior nodes represent branching decisions [12]. Thus, leaf nodes represent possible outcomes, while interior nodes outline the relationships among the features. Much like other models that represent software systems, (e.g., control flow graphs) the graphical representation of decision trees can be employed for software testing. The rationale behind our criteria is to exploit the way decisions are encoded into decision trees and apply a subset of mutation operators to the decision nodes (i.e., branching nodes). Then, evaluate the adequacy of the test data in a fashion that is similar to the way the test suite is evaluated for conventional programs when applying mutation testing. More specifically, the mutated tree models serve as a reference for selecting more effective test data, which can also be designed to effectively uncover incorrect classifications in decision tree models.

As mentioned, our approach is centered on applying mutation testing to decision tree models: the decision nodes are mutated through relational and constant mutation operators. Following this, we examine the dataset for instances where the classification provided by the mutated tree model diverged from that produced by the original decision tree. This process is performed in hopes of coming up with a dataset specifically tailored for killing the mutated tree models. In a manner akin to mutation analysis in traditional testing, new test cases are then generated, thereby enhancing the original dataset. Figure 1 illustrates this process. The resulting test set can then be used to evaluate other models training with the original dataset.

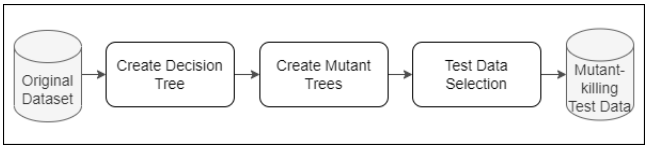

Figure 1: An overview of the proposed mutation-based approach to testing ML-based models.

We conducted an experimental study to evaluate the effectiveness of test data produced by our approach. We compare our approach with k-fold cross-validation, a common technique for assessing ML models. We explored whether our approach could improve the choice of a high-quality test dataset. The contributions presented in this paper are as follows:

- We propose an approach that applies mutation testing to decision tree models. This approach uses relational and constant mutation operators to mimic faults in the internal representation of decision tree models;
- We demonstrate how this approach may be used to choose test data samples that are more useful than randomly selected test data. Here, effectiveness is defined by how much the test data can decrease prediction scores.

The remainder of this paper is divided into the following sections. Related work is presented in Section 2. Our strategy is described in Section 3. The experimental strategy we employed to answer our research question is covered in Section 4. Section 5 covers the statistical analysis of the experiment's outcomes. Section 6 presents a discussion of the results from our experiment. Threats to validity are covered in Section 7, and concluding remarks are presented in Section 8.

2 RELATED WORK

While investigating software testing works for machine learning (ML), we observed that approaches utilizing mutation testing constituted a significant portion of these studies. [4, 23]. Furthermore, we found a study that critically reflects the use of mutation testing for ML [17]. This article argues that existing works that adapt mutation testing to Deep Learning (DL) systems do not always clearly show how mutation testing was adapted. In the following, we discuss some papers cited in this reflection paper [9, 11, 15, 20].

MuNN is a method for mutation analysis of neural networks [20]. It provides five mutation operators designed to build deep-learning models with potential bugs. Given a test set, the mutation score can be calculated as the fitness of the test. The mutation results can guide test generation and neural network test optimization. The experimental outcomes show some mutation analysis features different from conventional software. This is shown as an incentive to design some domain-dependent mutation operators instead of common mutation operators. Furthermore, some evidence is presented that the depth of the neuron is a sensitive factor in mutation analysis.

As MuNN, the DeepMutation is also a framework for applying mutation testing techniques to DL systems [15]. This paper proposes a source-level mutation testing technique for training data and training programs. To do this, a set of source-level mutation operators are designed. A model-level mutation testing technique is also proposed, defining a set of mutation operators that inject faults directly into DL models, and mutation testing metrics are proposed to measure test data quality. A continuation of this study is found in the Deepmutation++ framework [9], a mutation testing framework for Feed-Forward Neural Networks (FNNs) and Recurrent Neural Networks (RNNs). DeepMutation++ incorporates eight model-level operators for FNN models introduced in DeepMutation [15] and proposes nine new specialized operators for RNN models.

We found another work that investigates mutation operators for DL [11]. It was identified which configurations make them non-equivalent and non-trivial. In this paper, the authors propose a new definition of killed mutants that considers the stochastic nature of

DL systems. They compare this definition to the threshold-based drop in accuracy and show that this can be inconsistent in different runs.

DeepCrime [10] also proposes specific mutation operators for DL systems, however, this is the first tool that implements a set of DL mutation operators based on real DL faults. In this paper, the authors define 35 DL mutation operators and implement 24 DL mutation operators in DeepCrime, the first source-level pre-training mutation tool based on real DL faults. To evaluate the tool, they compared the sensitivity of the tool to changes in the test data quality with that of DeepMutation++ [9]. The results show that DeepCrime produces killable and non-trivial mutants, can effectively discriminate a weak test set from a strong one, and significantly outperforms the DeepMutation++ tool in these aspects.

In DeepMetis [18], the authors describe an automated way to increase existing test sets with inputs that kill mutants generated by DeepCrime [10]. The goal is to increase the mutation score of a test set by generating new entries that kill mutants not killed by the original test set. DeepMetis is a search-based test generator for DL systems that uses mutation fitting as a guideline. An experiment is performed to evaluate the approach, the results showed that DeepMetis is effective in increasing the given test set, increasing its ability to detect mutants by 63% on average. The experimental evaluation shows that the increased test suite can expose invisible mutants, which simulate the occurrence of undetected faults. DeepMetis differs from existing approaches in that its goal is to increase the ability to kill mutations in a test set. With the advent of DL mutation frameworks such as DeepMutation [15], MuNN [20] and DeepCrime [10], the problem of achieving a high mutation score is increasingly important, especially when mutants simulate real faults, as is the case with DeepCrime [10].

A recent study presents MTUL, a new mutation testing technique for Unsupervised Learning (UL) systems [14]. The authors present how they designed a series of mutation operators to simulate the unstable situations and possible errors that UL systems may encounter and define the corresponding mutation scores. They combine this technique with an autoencoder to generate contradictory samples and demonstrate the technique's feasibility based on three datasets.

Another recent Deep Learning focused paper proposes a Probabilistic Mutation Testing (PMT) approach that allows for a more consistent decision on whether a mutant is killed, through evaluation using three models and eight mutation operators [21]. The authors also analyze the trade-off between the approximation error and the cost of the method, showing that a relatively small error can be achieved for a manageable cost. The authors argue that PMT is the first step in this direction that effectively removes the lack of consistency between test runs of previous methods caused by the stochasticity of DNN training.

When searching for what is being discussed about mutation testing and ML we also found papers that discuss the use of ML applied to improve some aspects of mutation testing, but this is not the focus of our work.

The motivation for using tree models for test data selection is the representation power of these models. In addition, tree models can guide the tester with information about how much the tree-decision model was covered. This is an advance over manual ad hoc testing of ML-based software commonly used to test this kind of software. Therefore, two testing criteria for coverage of the decision-tree model have been proposed:

Decision Tree Coverage (DTC) criterion: Given a decision tree model M, a test set T is considered DTC-adequate if it includes tests that pass the tree from its root to all leaf nodes at least once.

Boundary Value Analysis (BVA) criterion: Test cases are designed to cover valid boundary values of decision nodes. Specifically, when designing test cases, this criterion requires the selection of values that explore either the lower or the upper boundary of each decision.

Therefore, these criteria are premised on the notion that the internal structure of decision tree models can help testers select test data. As an additional benefit of exploring the internal structure of models, the testing criteria also allow testers to quantify the effectiveness of test data by analyzing the number of outputs/decisions covered by such test data. According to the experimental results of our previous work, these adequacy testing criteria can be used to select more effective test data than randomly selected training data.

Motivated by the preliminary results, in this paper, we continue exploring the structure of decision tree models, now exploring mutation testing as an incremental strategy to select test data for ML applications.

In contrast to the previously mentioned research, which explored the use of mutation testing in the area of deep learning, our approach focuses on the application of mutation testing in classification models. Inspired by study which investigated the structure of decision trees to develop coverage criteria [19], our work is also based on the exploration of this structure for the generation of mutants. From these mutants, we identify the dataset instances that can inactivate them. This approach seeks to improve the robustness and reliability of classification models. By applying mutation testing on classification models, we can reveal hidden vulnerabilities and weaknesses to strengthen and improve the overall performance of algorithms.

3 DECISION TREE MUTATION TESTING

Mutation Testing identifies the most recurrent defects committed during development in traditional software testing. Applying small changes to the software under test encourages the tester to produce test cases that reveal the defects inserted in mutant programs, improving the quality of the test case set [5]

Mutants are created through mutation operators defined to specific programming languages or specification techniques. These operators aim to simulate the most common mistakes within the language or technique. Therefore, mutation operators consist of rules that specify the alterations to be made in the program being tested [5]. They are designed to introduce simple syntactic changes that reflect typical mistakes made by programmers or to fulfill specific testing objectives, such as ensuring the execution of every program edge [2].

To define our Decision Tree Mutation Testing (DTMT), we were inspired by the mutation operators defined for the C language [1]. These operators are divided into 4 groups: 1) statement mutation operators, 2) operator mutation operators, 3) variable mutation

operators, and 4) constant mutation operators. In particular, the following mutation operators are used in our definition:

(1) ORRN operator: this operator replaces every occurrence of a relational operator (<, >, <=, >= and ==) by another possible relational operator. Example: original: $a < b$; Mutant: $a <= b$.
(2) Cccr operator: this operator replaces every occurrence of a constant with another possible constant. Example: Given a set of constants: [7.5, 2.3, 3.14]; original: $a = 7.5$; Mutant: $a = 3.14$.

Based on these mutation operators, the DTMT is defined as follows: given a decision tree model, a test set is considered suitable for DTMT if it includes tests that generate a different classification result between the original and all mutant tree models. The mutant tree models are mutations of a tree model representing an ML application in which the operators ORRN and Cccr are applied in the intermediate/decision nodes of the original tree model.

In DTMT, each mutant tree is a test requirement, which, to be satisfied, is necessary to find data set rows that generate a different classification between the mutant and the original model.

The number of mutant trees depends on the number of intermediate nodes in the original tree. Thus, when applying the ORRN operator, we have 4 mutant trees for each node, one for each possible relational operator. For instance, consider the decision tree in Figure 2. This tree was generated from the Iris dataset [1]. In this tree, 8 nodes have relational operators; therefore, it is possible to generate 32 mutant decision trees for ORRN operator. In Figure 3, we have an example of one mutant decision tree, where the mutated node is highlighted.

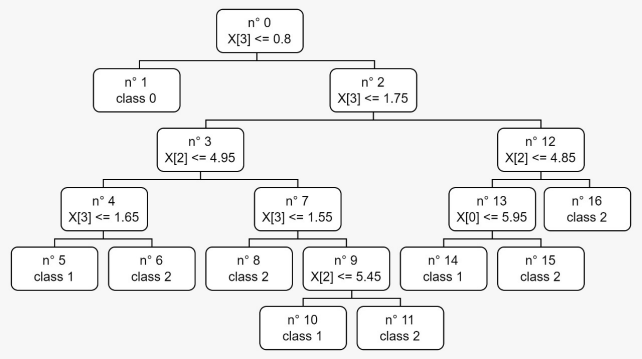

Figure 2: Decision tree model generated from the Iris dataset. The features depicted in the figure correspond to: x[0] = sepal length, x[1] = sepal width, x[2] = petal length, x[3] = petal width.

To apply the Cccr operator, each tree node containing a comparison between a feature and a constant value is selected, and the constant value is changed by another constant value used in another comparison with the same feature. The mutation is applied to all decision nodes in our approach; however, we have imposed a limitation of two changes per feature to prevent an overwhelming number of mutants. It has been observed that the number of

[1]https://www.kaggle.com/datasets/uciml/iris

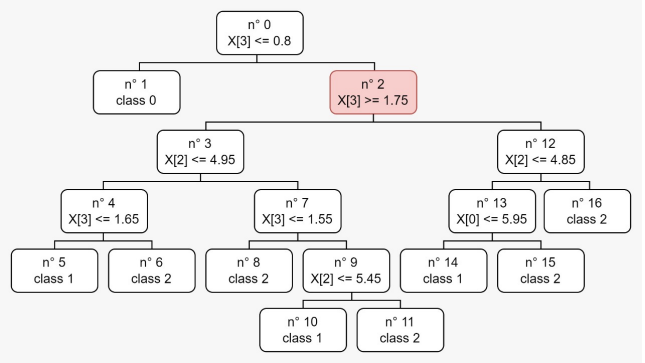

Figure 3: Example of a mutation for the ORRN operator applied in a tree node.

tree mutants generated can become impractical depending on the number of features present in the decision tree.

In Figure 4 we have an example of one mutant decision tree generated by Cccr operator. Considering that the decision tree has 8 nodes with comparison, 8 different constant values, and applying two mutations for each node, a total of 10 tree mutants are generated by the Cccr operator.

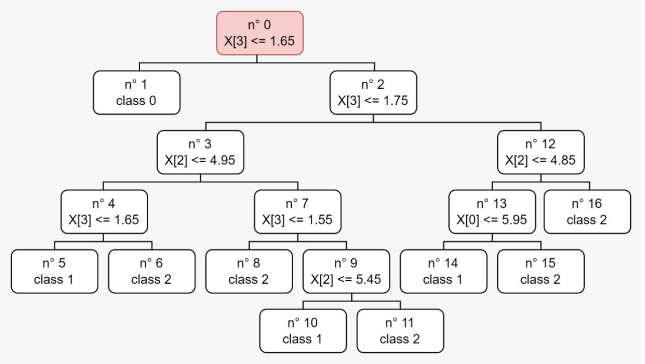

Figure 4: Example of a mutation for the Cccr operator applied in a tree node.

Like the DTC and BVA testing criteria, the DTMT can be applied for the generation of test data. From the point of view of test requirements, unlike DTC and BVA test case generation approaches which aim to satisfy the same number of test requirements: all paths from root to leaf in the tree under test. Our DTMT aims to kill all generated mutants.

We apply the DTMT to the canonical dataset of the Iris flower [7]. Applying the decision tree algorithm to the dataset resulted in a tree model with nine leaf nodes and eight decision nodes, as shown in Figure 2. Using the DTMT to guide the generation of test cases, we generated a set of 10 test cases. Since each test case can kill more than one mutant, we do not necessarily need to have the number of test cases equal to the number of test requirements.

To show how a test case can be considered capable of killing a mutant tree, we use the original tree presented in Figure 2, and

the mutant tree presented in Figure 3. In this mutation, node n°2 had its value changed from n°2 <= 1.75 to n°2 >= 1.75. Considering a test case in which X[3] = 1 in the original tree, the path would follow to the left child node (n°3) and would end at leaf node n°5, whereas in the mutant tree, it would be directed to the right child node (n°12) and would end at leaf node nº 16. We obtained different classification results and considered that the mutant was killed.

4 EXPERIMENTAL EVALUATION

To evaluate the effectiveness of our Decision Tree Mutation Testing approach, we set out to compare it with the cross-validation approach. We hypothesize that applying mutation to decision tree models tends to result in higher-quality test suites, that is, test suites capable of finding more problems (faults) in ML applications.

We designed the experiment to answer the following research question (RQ):

RQ: *How does the effectiveness of the test suites derived from DTMT compare to the random test cases from 10-fold cross-validation?*

4.1 Scoping

The scope of our experiment is defined by setting its goals, which is based on the GQM template [22] as follows:

Analyze DTMT approach
for the purpose of evaluation
with respect to its effectiveness
from the point of view of the researcher
in the context of testing ML-based programs.

4.2 Hypotheses Formulation

To enable the application of statistical tests, we further refined our RQ into specific hypotheses. We defined our prediction for **RQ** as: our mutation-based criterion is more effective than random testing. Thus, our RQ was turned into the following hypotheses:

Null hypothesis, H_0: there is no difference in effectiveness between DTMT and random testing.

Alternative hypothesis, H_1: there is a significant difference in effectiveness between DTMT and random testing.

Our main goal is to investigate whether DTMT leads to selecting more effective test data. To evaluate this assumption, we set out to derive a test suite that satisfies DTMT when applied to a decision tree model. Subsequently, we use the resulting DTMT-adequate test suite to assess the performance of models generated from other machine learning algorithms, such as k-Nearest Neighbors (k-NN). Specifically, we first train models using the k-NN algorithm on the original dataset (i.e., the training data); these models are both trained and tested using 10-fold cross-validation. We then compare the results from this 10-fold cross-validation with the outcomes from applying the DTMT test suites. Figure 5 provides an overview of this process.

In the context of traditional software testing, the effectiveness of a criterion is determined by its ability to generate test inputs that can reveal more faults. Hence, a criterion C_1 is deemed more effective than another criterion C_2 if the former leads to test inputs

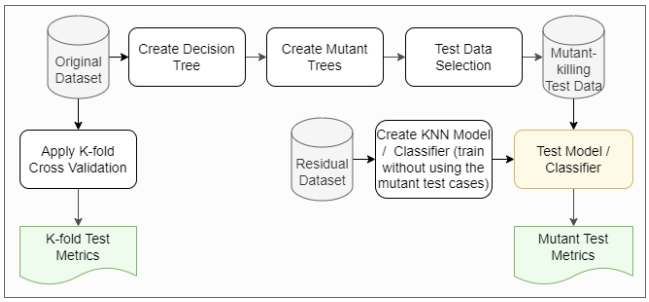

Figure 5: Experiment flow illustration.

that can uncover more faults than those derived from applying the latter. This concept also applies to our study. For a given model M, if the performance is poorer with test inputs derived from C_1 than those generated from C_2, then we say that C_1 is more effective than C_2. Given a metric, denoted as f, to gauge the quality of model M and a test suite, denoted as T. The score of running test suite T against model M, according to metric f, is expressed as $f(M(T))$. Therefore:

$$f(M(T_1)) < f(M(T_2))$$

indicates that T_1 is more effective than T_2 according to f because T_1 revealed more cases in which M *fails* than T_2 did.

Many approaches have been devised to evaluating and comparing ML models. In our comparative analysis, we decided to take into account several multiple widely used ML performance metrics. As such, we selected precision, recall, accuracy, and F_1 metric. Thus, the dependent variable that is key in answering our RQ is *effectiveness*. Within the framework of this experimental study, effectiveness is quantified using the ML performance metrics mentioned previously.

In this experiment, we focus on a single factor with two distinct treatments. The factor under consideration is the testing approach, while the treatments are our adequacy criteria and random testing (i.e., 10-fold cross-validation).

4.3 Instrumentation

Prior to conducting our experiment, we created experimental objects. These objects are primarily divided into two categories: Python scripts for loading and processing datasets, including test case generation, and scripts for the analysis of results. We employed Google Colaboratory (also known as Colab)[2], a cloud-based environment that facilitates the execution of Python scripts via a browser. Google Colab enables users to create notebooks, which are essentially Jupyter notebooks, combining rich text and executable Python code within a single document.

For data handling and analysis, we utilized the `pandas`[3] library. All the ML algorithms were implemented using `scikit-learn`, a leading Python library for machine learning [16]. To store Predictive

[2]https://colab.research.google.com/
[3]https://pandas.pydata.org

Model Markup Language (PMML)[4] files, which were employed for generating the mutant decision trees, we used Google Drive.[5]

4.4 Execution

To assess the effectiveness of our mutation-based approach, we chose a sample of 12 publicly available datasets. These datasets are widely utilized for training machine learning models. We selected these particular datasets due to their simplicity compared to others. Furthermore, these datasets are frequently applied to solve more straightforward problems; consequently, inputting these datasets into a DT algorithm yields smaller, more interpretable trees. A summary of the datasets used in our experiment is provided in Table 1.

Table 1: Overview of the datasets used in the experiment.

Datasets	Samples	Attributes	Classes	Samples per Class
Banknote Authentication	1,372	5	2	[762, 610]
Breast Cancer Wisconsin	569	31	2	[212, 357]
Cleveland Heart Disease	297	15	2	[160, 137]
Haberman's Survival	306	4	2	[225, 81]
Ionosphere	351	35	2	[225, 126]
Iris	150	5	3	[50, 50, 50]
Mammography	11,183	7	2	[10,923, 260]
Oil Spill	937	50	2	[896, 41]
Phoneme	5,404	6	2	[3,818, 1,586]
Pima Indians Diabetes	768	9	2	[500, 268]
Sonar, Mines vs. Rocks	208	61	2	[97, 111]
Wine Recognition	178	14	3	[59, 71, 48]

We loaded all datasets into the Google Colab (i.e, Python) environment. To this end, we used `pandas` library's methods for loading external data. Several datasets had `NA` values, so we performed some data cleansing (also employing the `pandas` library) to create more consistent datasets. After tidying up the data, we split the datasets into two parts: training data (i.e., `X`) and their corresponding outputs or labels (i.e., `y`).

The next step was to build DT models from the training data. To fit the models to the training data we used `scikit-learn`, which employs the classification and regression tree (CART) algorithm to train decision trees [8]. DTs in `scikit-learn` are implemented in such a way that the resulting models also contain auxiliary information that we used in later steps of the experiment execution. Specifically, these models also keep information regarding all nodes, including the rules in decision nodes that resemble `if-else` code rules from conventional programming languages.

After fitting each DT model to the respective complete dataset, the resulting models are stored in a XML-based format: predictive model markup language (PMML). The PMML models for each classifier are then used in the mutation generation step. Given the XML-based structure of PMML files, the parsing of each PMML model starts at the root and decision nodes are mutated as the tree is traversed. To extract model information from the resulting XML Schema in PMML files, we used BeautifulSoup, a Python library for parsing XML and HTML files.

Once mutant DTs have been generated, our approach randomly selects instances (i.e., rows) from the dataset and then compares the predictions generated by the original tree and the prediction of each mutant tree. Specifically, our approach randomly walks through the dataset for each mutant tree until it finds an instance that kills the mutant under analysis.

This focus on model predictions (i.e., outcomes) to compare DTs emphasizes the selection of instances that contain values that cause mutant trees to make predictions that differ from the predictions made by the original tree model: mutants that result in different predictions are deemed *dead*, while those that generate predictions matching those of the original model are classified as *alive*.

This thorough exploration of the dataset can determine whether there is at least one instance that can kill the live mutant tree under analysis or, conversely, whether the dataset lacks any such instance. If at least one instance can kill the mutant, that instance is chosen. If no such instance exists, the mutant tree under analysis is deemed equivalent to the original tree. By selecting instances that kill mutants and after excluding duplicates, our approach yields a subset of the dataset containing mutant-killing instances (which we assume is a high-quality test dataset and a good starting point for validating other ML classification models).

We initially used the original datasets to train and test k-NN models, employing a 10-fold cross-validation process. Subsequently, we generated new k-NN models that were trained on data from the original datasets, that is, excluding the data selected for the test set generated by mutation (i.e., a subset of the dataset that does not contain instances that kill mutants). The outcomes of these tests were then compared with the results from the previously run 10-fold cross-validation.

5 EXPERIMENTAL RESULTS

In this section, we examine the effectiveness of our mutation-based approach by looking at the performance of models when they are cross-validated 10 times versus when tested with our test sets that kill mutants.

5.1 Objects Characteristics

Our experiment considered 6,634 mutant DT models generated from the 12 chosen datasets. Table 2 shows an overview of the mutants generated for each dataset. The table presents, for each dataset, the amount of non-terminal DT nodes, the number of relational mutants created (under ORRN Mut), the number of constant mutants generated (under Cccr Mut), and the total number of mutants generated (under Total Mut). It also includes the quantity of test data used to kill the relational (ORRN Test Data) and constant mutants (Cccr Test Data). Additionally, it lists the total volume of test data, which is the combination of "ORRN Test Data" and "Cccr Test Data" after removing duplicates. Furthermore, it provides the proportion of the resulting test set concerning the number of instances in the original dataset (represented as Test Data/Dataset) and, finally, the amount of mutants that have not been killed (Live Mut) and the Score Mutation (Score Mut). As shown in Table 2, most mutants were generated from the DT model of the Phoneme dataset (totaling 3,116 mutants). The Iris dataset led to the simplest tree model, which resulted in 42 mutants.

Given the outliers in our data regarding the number of mutants and test data generated, we consider the median values to be a

[4]https://www.ibm.com/docs/pt-br/db2/11.1?topic=analytics-pmml-markup-language-data-mining
[5]Experiment repository: https://shorturl.at/bxAX0

Table 2: Overview of data generated in the experiment.

Datasets	Decision Nodes	ORRN Mut	Cccr Mut	Total Mut	ORRN Test Data	Cccr Test Data	Total Test Data	Test Data/Dataset (%)	Live Mut	Score Mut
Banknote Authentication	26	104	44	148	33	26	45	3.28	29	80.41
Breast Cancer Wisconsin	21	84	14	98	24	11	29	5.10	23	76.53
Cleveland Heart Disease	100	400	148	548	79	75	103	34.68	118	78.47
Haberman's Survival	101	404	187	591	69	92	111	36.27	116	80.37
Ionosphere	22	88	14	102	22	7	26	7.41	26	74.51
Iris	8	32	10	42	7	6	10	6.67	8	80.95
Mammography	159	636	306	942	151	165	205	1.83	173	81.63
Oil Spill	34	136	26	162	33	17	39	4.16	38	76.54
Phoneme	521	2084	1032	3116	516	465	624	11.55	532	82.93
Pima Indians Diabetes	126	504	235	739	107	113	156	20.31	140	81.06
Sonar, Mines vs. Rocks	22	88	4	92	20	3	21	10.10	22	76.09
Wine Recognition	11	44	10	54	12	8	14	7.87	11	79.63
	Descriptive Statistics									
Max	521	2,084	1,032	3116	516	465	624	36.27	532	82.93
Min	8	32	4	42	7	3	10	1.83	8	74.51
Mean	95.92	383.67	169.17	552.83	89.42	82.33	115.25	12.44	103	79.09
Median	30	120	35	155	33	21.5	42	7.64	33.5	80.00
Std Dev	143.28	573.13	290.86	863.73	141.23	131.50	171.91	11.79	146.72	2.62

more accurate measure of central tendency than the mean. Thus, on average (median), the models in our experiment have 155 mutants, of which approximately 120 correspond to operator mutation and 35 to constant mutation. Regarding the size of our test set, we have an average of 42 test data for each dataset, which corresponds to 7.64% of the original dataset. As for the number of mutants that were not killed, we have an average of 33.5, and this implies that using our test dataset, we achieved a median mutation score of 79.09%.

As each mutant represents a test requirement, the amount of test cases generated by our criteria is proportional to the number of decision nodes in a given model. Therefore, applying our criteria to the resulting models, on average (median), approximately 155 test requirements.

5.2 Hypothesis Testing

To investigate whether DTMT leads to the selection of test data that are more effective in evaluating the performance of ML models, we used parametric tests (i.e., two-tailed unpaired two-sample t-test) to assess differences in the mean value of the metrics used in our experiment (i.e., precision, recall, accuracy, and F_1) [19].

We checked for normality using the Shapiro-Wilk test before running the tests. According to the results of the Shapiro-Wilk test, all distributions of the results of applying the DTMT test data are normal. Nevertheless, the results of the Shapiro-Wilk test also show that the Precision results of the 10-fold cross-validation approach significantly deviate from normality: W = 0.843, p=0.030 (as shown in Table 3). Hence, we employed a non-parametric (Wilcoxon signed-rank) test to analyze this metric.

For recall, accuracy, and F_1, we used two-tailed unpaired two-sample t-tests. This parametric test is a well-known approach to comparing samples (i.e., datasets) measured on the same dependent variables (i.e., chosen performance metrics) but under different levels of an independent variable (i.e., DTMT and 10-fold cross-validation). We used a non-parametric test (Wilcoxon signed-rank test) for the metric precision, as the Shapiro-Wilk test indicated, a deviation from normality. According to the test results, our criteria can generate test data that differ significantly from the 10-fold cross-validation test data in most cases. The results suggest that the DTMT test data differ significantly from the original data on almost all metrics but precision. Results are represented by W-statistic, t-scores and p-values in Table 4.

6 DISCUSSION

To answer our **RQ**, we examined the feasibility of using our mutation-based approach to drive test case selection for ML models. We aimed to select entries that killed as many mutants as possible, given that each mutant is a test requirement. The overarching motivation for this is to arrive at a set of test cases that negatively impact the performance of the model under test. We postulate that it is possible to select stronger test data when we aim to kill the mutants generated by taking advantage of the internal structure of decision tree models. In this context, we measure how *effective* the test input is for an ML model against the data the model was trained on: Effectiveness was assessed by the impact of test data on the predictive performance of the model under evaluation. The more detrimental the test data was to the model's performance, the more effective it was considered. Here, random testing is the 10-fold cross-validation, a technique that involves dividing the dataset into 10 parts and repeating the model training and testing 10 times, resulting in a general performance estimate.

According to the results shown in Table 2, Table 3 and Figure 6, in the evaluation of k-NN models using our generated test data, we observed a significant decrease in predictive performance across nearly all evaluated metrics. Our response to **RQ** centers on the metrics mentioned in Subsection 4.2, which were used as proxies for the effectiveness of the resulting test data. Considering the values of F_1, the results suggest that the test data generated using our approach leads to more challenging inputs, resulting in a significant decrease in performance compared to the baseline (F_1 10 fold cross-validation).

For example, for the k-NN model generated from the `Oil Spill` dataset, during 10-fold cross-validation, the model yielded an F_1 value of 0.94, it turns out that this model performed significantly worse when evaluated against MTDT test data: 0.46. For the k-NN model generated from the `Ionosphere` dataset, the F_1 value for the cross-validation was 0.84 and there was a significantly more pronounced decrease in performance when the model was tested

Table 3: Results obtained by the evaluated metrics for the DTMT approach and 10-fold cross-validation.

Dataset	Mutation Based Approach				10-fold Cross-Validation			
	Precision	Recall	Accuracy	F_1	Precision	Recall	Accuracy	F_1
Banknote Authentication	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
Breast Cancer Wisconsin	0.71	0.69	0.69	0.69	0.93	0.93	0.93	0.93
Cleveland Heart Disease	0.13	0.29	0.29	0.17	0.35	0.48	0.48	0.40
Haberman's Survival	0.62	0.62	0.62	0.57	0.66	0.69	0.69	0.67
Ionosphere	0.80	0.50	0.50	0.45	0.87	0.85	0.85	0.84
Iris	0.73	0.70	0.70	0.71	0.97	0.96	0.97	0.96
Mammography	0.70	0.70	0.70	0.67	0.99	0.99	0.99	0.99
Oil Spill	0.39	0.56	0.56	0.46	0.93	0.95	0.95	0.94
Phoneme	0.64	0.64	0.64	0.63	0.89	0.90	0.90	0.89
Pima Indians Diabetes	0.48	0.49	0.49	0.48	0.68	0.68	0.68	0.68
Sonar, Mines vs. Rocks	0.69	0.62	0.62	0.59	0.84	0.83	0.83	0.82
Wine Recognition	0.71	0.50	0.50	0.49	0.70	0.69	0.69	0.67
	Descriptive Statistics and Tests of Normality							
Max	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
Min	0.13	0.29	0.29	0.17	0.35	0.48	0.48	0.40
Mean	0.63	0.61	0.61	0.57	0.82	0.83	0.83	0.82
Std Dev	0.22	0.17	0.17	0.20	0.19	0.16	0.16	0.18
Shapiro-Wilk (W)	W=0.908, p=0.202	W=0.925, p=0.330	W=0.926, p=0.336	W=0.939, p=0.486	W=0.843, p=0.030	W=0.885, p=0.102	W=0.885, p=0.102	W=0.871, p=0.068

with DTMT test data (0.45). In terms of F_1, there is a statistically significant difference between DTMT (mean = 0.57) and 10 fold cross-validation (mean = 0.82): W = -3.12, p-value = 0.005 (Table 4).

The results presented in Table 3 and Figure 6, demonstrate a statistically significant difference between DTMT and 10-fold cross-validation across all evaluated metrics. This suggests that DTMT can be employed as effective approach to generate test data from existing datasets to detect erroneous behavior (i.e., failure to predict multiple test inputs) in ML models.

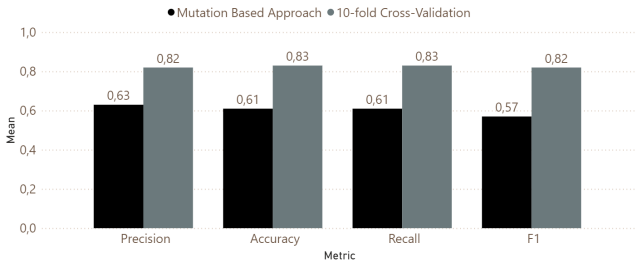

Figure 6: Comparison of the results obtained for the evaluated metrics to DTMT and 10-fold cross-validation

Table 4: Summary of the results from the statistical tests.

	Paired Samples Test (t)
Metric	**DTMT x Cross-validation**
Accuracy	t = -3.24, p-value = 0.003
Recall	t = -3.23, p-value = 0.003
F_1	t = -3.12, p-value = 0.005
	Wilcoxon Signed-Rank Test (W)
Metric	**DTMT x Cross-validation**
Precision	W = 23, p-value = 0.374

7 THREATS TO VALIDITY

Throughout our experiment, we took several precautions to alleviate potential threats to the validity of our experiment and the findings thereof. However, as is typical with most experimental studies, it is impossible to remedy all threats to validity completely. A potential threat to validity is that we employed the same datasets and metrics used in our previous research. As a result, all threats associated with these elements also carry over into our study.

The datasets may potentially threaten external validity as they may not adequately represent the target population. Our chosen datasets are smaller and cleaner than those commonly encountered in practice. Therefore, we cannot dismiss the possibility that the results could have differed if larger datasets had been selected. The measures used in the experiment may be considered a potential threat to construct validity as they may not be adequate to assess the effects we set out to investigate. Specifically, precision, recall, accuracy, and F_1 score may not be key predictors of test data suitability for ML-based programs. However, it is worth mentioning that these four measures are widely used to evaluate ML models, which mitigates the risk of this threat.

A fundamental limitation of our approach is that it assumes that only numerical values appear at decision nodes. In addition, the exhaustive search in the original dataset is currently done sequentially, which impacts performance when creating our set of test cases.

8 CONCLUDING REMARKS

This paper presents a new proposal for mutation testing for ML systems modeled by a decision tree called DTMT. The use of the decision tree model is attractive due to its simplicity and representation power. In the past, mutation testing was investigated in the context of specification models, showing that it is interesting to explore this testing criterion for software models [13].

This proposal is based on the traditional mutation testing technique, whose aim is to evaluate the quality of test cases of a program, verifying its ability to detect defects. This is done by introducing

mutations, that is, modified versions of the original program with specific faults.

Our approach is based on the premise that by using the internal structure of decision tree models to create mutant trees and using these mutations to select test data that kill these mutants, we can create a set of test cases that are more effective than the randomly selected ones.

The experimental results indicate that DTMT is a promising approach for generating test data to detect incorrect behaviors in ML systems.

We believe our work can motivate new propositions of testing techniques that explore the internal structure of ML models.

In future research, we intend to perform a cost analysis of our mutation testing approach applied to decision trees and investigate its effectiveness in testing different ML models. Also, we intend to explore random forest models instead of decision trees to improve the quality of test data. Random forests are a technique that combines multiple decision trees to mitigate overfitting and enhance prediction accuracy

ACKNOWLEDGMENTS

Beatriz N. C. Silveira was funded by CNPq (Conselho Nacional de Desenvolvimento Científico e Tecnológico), process number 133469/2020−4. Sebastião H. N. Santos was funded by CAPES (Coordenação de Aperfeiçoamento de Pessoal de Nível Superior). The authors also thank the support of FAPESP, process number 2019/06937−0, and CNPq, under processes number 308636/2021−0 and 308445/2021−0.

REFERENCES

[1] Hiralal Agrawal, Richard A. Demillo, Bob Hathaway, William Hsu, Wynne Hsu, E. W. Krauser, R. J. Martin, Aditya P. Mathur, and Eugene H. Spafford. 1989. *Design Of Mutant Operators For The C Programming Language.* W. Lafayette, IN 47907, Software Engineering Research Center Department of Computer Sciences Purdue University.

[2] Paul Ammann and Jeff Offutt. 2016. *Introduction to software testing.* Cambridge University Press.

[3] Maurício Aniche, Erick Maziero, Rafael Durelli, and Vinicius H. S. Durelli. 2022. The Effectiveness of Supervised Machine Learning Algorithms in Predicting Software Refactoring. *IEEE Transactions on Software Engineering* 48, 4 (2022), 1432–1450.

[4] Houssem Ben Braiek and Foutse Khomh. 2020. On Testing Machine Learning Programs. *Journal of Systems and Software* 164 (2020), 110542.

[5] R.A. DeMillo, R.J. Lipton, and F.G. Sayward. 1978. Hints on Test Data Selection: Help for the Practicing Programmer. *Computer* 11, 4 (1978), 34–41.

[6] V. H. S. Durelli, R. S. Durelli, S. S. Borges, A. T. Endo, M. M. Eler, D. R. C. Dias, and M. P. Guimarães. 2019. *Machine Learning Applied to Software Testing: A Systematic Mapping Study.*

[7] R. A. Fisher. 1936. The Use of Multiple Measurements in Taxonomic Problems. *Annals of Eugenics* 7, 2 (1936), 179–188.

[8] Aurélien Géron. 2019. *Hands-On Machine Learning with Scikit-Learn, Keras, and Tensorflow: Concepts, Tools, and Techniques to Build Intelligent Systems* (2nd ed.). O'Reilly. 856 pages.

[9] Q. Hu, L. Ma, X. Xie, B. Yu, Y. Liu, and J. Zhao. 2019. DeepMutation++: A Mutation Testing Framework for Deep Learning Systems. In *34th IEEE/ACM International Conference on Automated Software Engineering (ASE).* 1157–1161.

[10] Nargiz Humbatova, Gunel Jahangirova, and Paolo Tonella. 2021. DeepCrime: Mutation Testing of Deep Learning Systems Based on Real Faults. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2021).* Association for Computing Machinery.

[11] Gunel Jahangirova and Paolo Tonella. 2020. An Empirical Evaluation of Mutation Operators for Deep Learning Systems. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST).* 74–84.

[12] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. 2013. *An Introduction to Statistical Learning: with Applications in R.* Springer Texts in Statistics. 426 pages.

[13] Yue Jia and Mark Harman. 2010. An analysis and survey of the development of mutation testing. *IEEE transactions on software engineering* 37, 5 (2010), 649–678.

[14] Yuteng Lu, Kaicheng Shao, Weidi Sun, and Meng Sun. 2022. MTUL: Towards Mutation Testing of Unsupervised Learning Systems. In *Dependable Software Engineering. Theories, Tools, and Applications*, Wei Dong and Jean-Pierre Talpin (Eds.). Springer Nature Switzerland, Cham, 22–40.

[15] Lei Ma, Fuyuan Zhang, Jiyuan Sun, Minhui Xue, Bo Li, Felix Juefei-Xu, Chao Xie, Li Li, Yang Liu, Jianjun Zhao, and Yadong Wang. 2018. DeepMutation: Mutation Testing of Deep Learning Systems. In *2018 IEEE 29th International Symposium on Software Reliability Engineering (ISSRE).* 100–111.

[16] A. C. Müller and Sarah Guido. 2016. *Introduction to Machine Learning with Python: A Guide for Data Scientists.* O'Reilly Media. 400 pages.

[17] Annibale Panichella and Cynthia C. S. Liem. 2021. What Are We Really Testing in Mutation Testing for Machine Learning? A Critical Reflection. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER).* 66–70.

[18] Vincenzo Riccio, Nargiz Humbatova, Gunel Jahangirova, and Paolo Tonella. 2022. DeepMetis: Augmenting a Deep Learning Test Set to Increase Its Mutation Score. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE '21).* IEEE Press, 355–367.

[19] Sebastião Santos, Beatriz Silveira, Vinicius Durelli, Rafael Durelli, Simone Souza, and Marcio Delamaro. 2021. On Using Decision Tree Coverage Criteria for Testing Machine Learning Models. In *Proceedings of the 6th Brazilian Symposium on Systematic and Automated Software Testing.* 1–9.

[20] Weijun Shen, Jun Wan, and Zhenyu Chen. 2018. MuNN: Mutation Analysis of Neural Networks. In *2018 IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C).* 108–115.

[21] Florian Tambon, Foutse Khomh, and Giuliano Antoniol. 2023. A probabilistic framework for mutation testing in deep neural networks. *Information and Software Technology* 155 (2023), 107–129.

[22] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén. 2012. *Experimentation in Software Engineering.* Springer. 236 pages.

[23] J. M. Zhang, M. Harman, L. Ma, and Y. Liu. 2020. Machine Learning Testing: Survey, Landscapes and Horizons. *IEEE Transactions on Software Engineering (Early Access)* (2020), 1–37.